Hochschule
Bonn-Rhein-Sieg
University of Applied Sciences

FACHBEREICH
INFORMATIK

Programmierung 1

Einstieg in Java

WS 2026/27

Bachelor Cyber Security & Privacy (B.Sc.)

André Kless, M.Sc.

Inhalte dieser Vorlesung

Java und seine Ausführung

Quelltext, Compiler, Bytecode und Java Virtual Machine (JVM)

Das erste eigene Programm

Im Terminal übersetzen und starten, danach in IntelliJ IDEA arbeiten

Die bekannten Grundbausteine in Java

Variablen, Ein- und Ausgabe, Verzweigungen und Schleifen

Programme verstehen und prüfen

Abläufe verfolgen, Fehler finden und den ggT berechnen

Was ist Java?

Eine Programmiersprache

Wir formulieren Anweisungen in einem eindeutig geregelten Quelltext.

Statisch typisiert

Variablen besitzen einen festen Datentyp. Der Compiler prüft die Verwendung.

Objektorientiert

Java organisiert größere Programme mit Klassen und Objekten. Das folgt später.

Mit eigener Laufzeitumgebung

Die JVM führt Bytecode aus. Garbage Collection gibt nicht mehr benötigten Speicher frei.

Ausblick: ggT in Pseudocode und Java

Pseudocode

```
DEKLARIERE a, b
LIES a EIN
LIES b EIN
SOLANGE a ≠ b:
  WENN a > b:
    SETZE a AUF a - b
  SONST:
    SETZE b AUF b - a
  ENDE WENN
ENDE SOLANGE
GIB a AUS
```

Java

```
void main() {
    int a = Integer.parseInt(
            IO.readln("a: "));
    int b = Integer.parseInt(
            IO.readln("b: "));

    while (a != b) {
        if (a > b) {
            a = a - b;
        } else {
            b = b - a;
        }
    }

    IO.println(a);
}
```

Ausblick: ggT in Pseudocode und Java

Pseudocode

```
A  DEKLARIERE a, b                    D
   LIES a EIN                         D
   LIES b EIN                         D
B  SOLANGE a ≠ b:
C     WENN a > b:
         SETZE a AUF a - b            D
      SONST:
         SETZE b AUF b - a            D
      ENDE WENN
   ENDE SOLANGE
   GIB a AUS                          D
```

Java

```
void main() {
A     int a = Integer.parseInt(
              IO.readln("a: "));      D
      int b = Integer.parseInt(
              IO.readln("b: "));      D

B     while (a != b) {
C        if (a > b) {
            a = a - b;                D
         } else {
            b = b - a;                D
         }
      }

      IO.println(a);                  D
}
```

Legende: A Sequenz B Iteration C Selektion D Einzelanweisung

Java: historische Idee und heutiger Einstieg

Zeit	Entwicklung	Bedeutung
1991	Sun-Team um James Gosling	Programme für unterschiedliche Geräte
1995	Öffentliche Vorstellung von Java	Ein Programm auf mehreren Plattformen
2006/07	Öffnung als Open Source / OpenJDK	Gemeinsame Weiterentwicklung
2010	Oracle übernimmt Sun Microsystems	Oracle führt die Java-Entwicklung fort
2025	Java 25 als LTS-Version	Kompakter Einstieg mit `void main()`

Für diese Lehrveranstaltung verwenden wir Java 25 LTS.

LTS = Long-Term Support. Java und JavaScript sind eigenständige Sprachen.

Compiler und Interpreter

Aspekt	Compiler	Interpreter
Aufgabe	Code in eine andere Codeform übersetzen	Anweisungen lesen und ausführen
Eingabe	Quelltext oder Zwischencode (z. B. Bytecode)	Quelltext oder Zwischencode (z. B. Bytecode)
Ergebnis	Übersetzter Code, z. B. Bytecode oder Maschinencode	Wirkungen des Programms, z. B. Berechnung oder Ausgabe
Bei Java	`javac` übersetzt Quelltext in Bytecode.	Der Interpreter der JVM führt Bytecode aus.

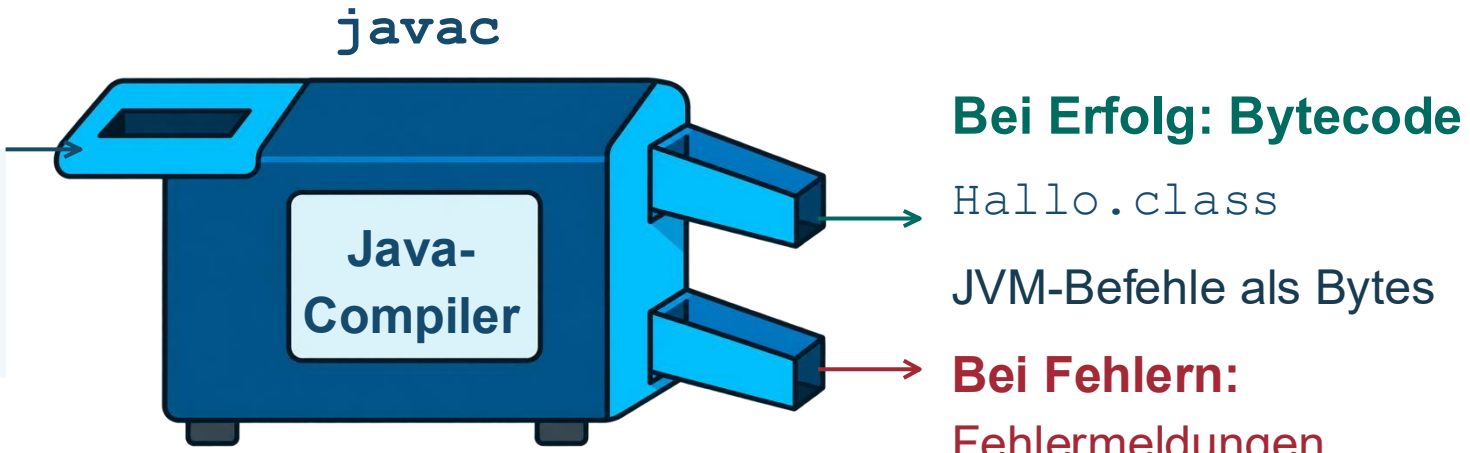
Ein Compiler übersetzt Code. Ein Interpreter führt Anweisungen aus.

Vor dem Start: Quelltext kompilieren

Quelltext

`Hallo.java`

```
void main() {
    IO.println("Hallo, Java!");
}
```



Bei Erfolg: Bytecode

`Hallo.class`

JVM-Befehle als Bytes

Bei Fehlern:

Fehlermeldungen

Kompilieren: prüfen und übersetzen

`javac Hallo.java` prüft die Syntax wie der Syntaxprüfer aus Kapitel 2.
Passen auch Namen und Typen, erzeugt der Compiler die Datei `Hallo.class`.

Das Programm läuft noch nicht

Der Bytecode in `Hallo.class` ist für die JVM bestimmt. Die CPU braucht Maschinencode.

Was der Java-Compiler prüft

```
int anzahl = 3;
```

Die Grammatik aus Kapitel 2 beschreibt zulässige Formen.

Zusätzliche Regeln bestimmen, ob Namen und Typen passen.

Java 25: offizielle Grammatik

1 Token erkennen

`int · anzahl · = · 3 · ;`

2 Syntax anhand der Grammatik prüfen

Entspricht die Deklaration der Java-Grammatik?

3 Statische Semantik prüfen

Passt der Wert zum Typ? Sind verwendete Namen bekannt?

4 Bytecode erzeugen

Die gültigen Anweisungen in JVM-Befehle übersetzen.

Korrekt übersetzt bedeutet noch nicht: Das Programm löst die richtige Aufgabe.

Bytecode und Maschinencode

Bytecode ist eine Folge von Befehlen für die JVM.
Er steht binär (als Bytes) in der Datei `Hallo.class`.

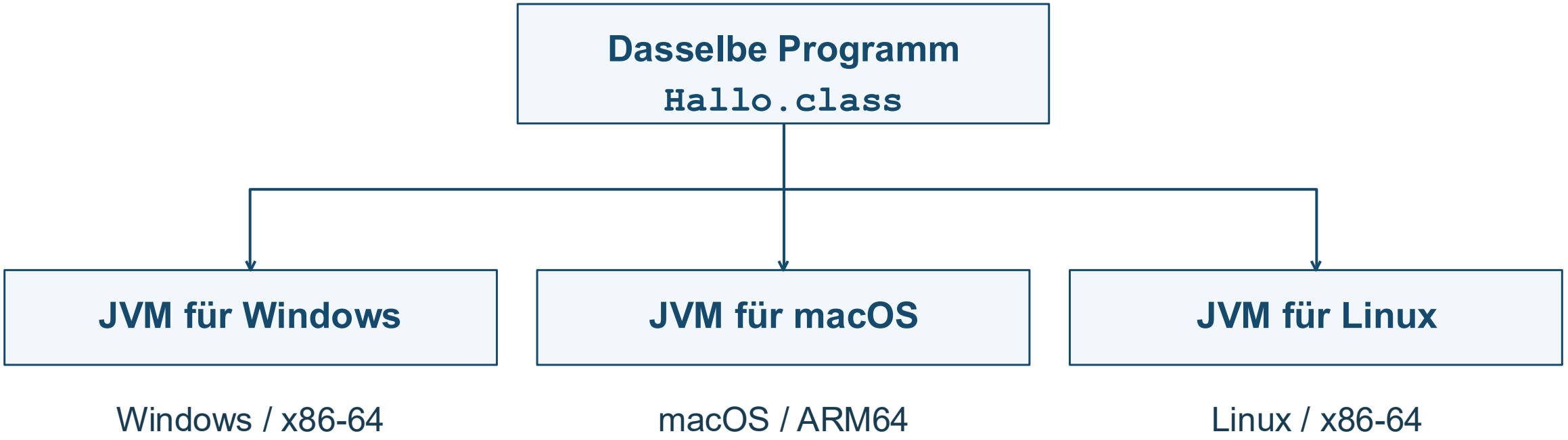
Darstellung	Beispiel	Bedeutung
Java-Quelltext	`IO.println("Hallo, Java!");`	Gibt den Text auf dem Bildschirm aus.

JVM-Bytecode	`ldc #7`
	`invokestatic #9`
	`return`
		Text „Hallo, Java!“ laden.
		Ausgabemethode aufrufen.
		`main` beenden.

Maschinencode	z. B. für ARM64 oder x86-64	Direkt vom jeweiligen Prozessor ausführbar.

Lesbarer Ausschnitt aus `main`, angezeigt mit `javap -c Hallo.class`.
`#7` und `#9` verweisen hier auf den Text und die Ausgabemethode in der Klassendatei.

Warum gibt es eine JVM?



Die JVM überbrückt Unterschiede zwischen Plattformen.

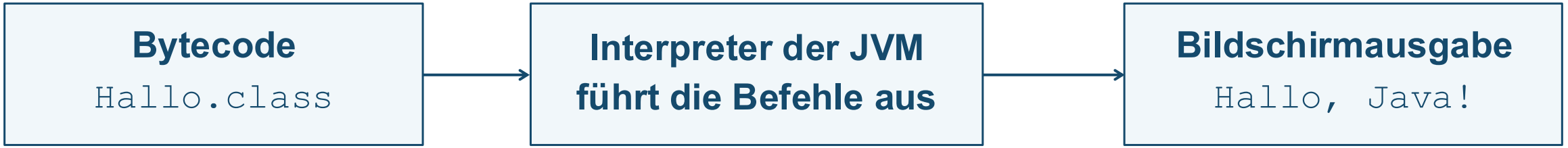
Voraussetzung: passende Java-Version und benötigte Bibliotheken.

Eine JVM bildet eine Ausführungsumgebung für Programme, keinen kompletten PC.

Beim Start: Bytecode interpretieren

```
java Hallo
```

Die JVM lädt die Datei `Hallo.class`.
Typischer Start mit der JVM:



Hier läuft das Programm bereits

Der Interpreter läuft als Programm auf der CPU. Der Bytecode ist seine Eingabe.
Er erkennt die Bytecode-Befehle und führt die dafür vorgesehenen Operationen aus.

Interpretiert wird der Bytecode aus der `.class`-Datei.

Wie Interpreter und CPU zusammenarbeiten

Der Interpreter ist ein Programm in Maschinencode, das auf der CPU läuft.
Er liest den Bytecode als Eingabe und wählt die passende Operation.

Bytecode als Eingabe

`iadd`

Addiere zwei ganze Zahlen.

Beispielwerte: 2 und 3

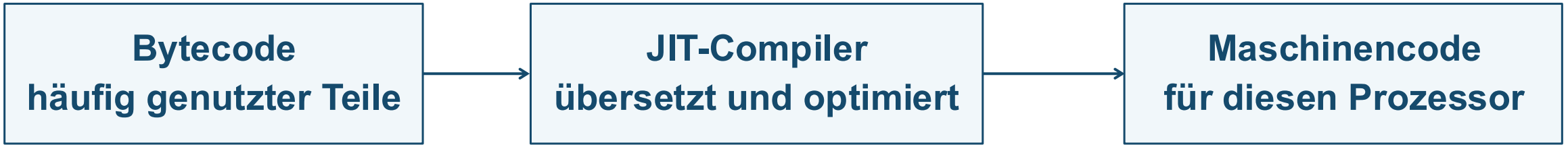
CPU führt das Interpreterprogramm aus

1. Der Interpreter erkennt `iadd`.
2. Die CPU führt die vorhandenen Maschinenbefehle für die Addition aus.
3. Ergebnis: `5`

Beim Interpretieren nutzt die CPU vorhandenen Maschinencode des Interpreters.

Zur Laufzeit: häufige Teile kompilieren

JIT = Just in Time: Übersetzen während der Programmausführung.
Die JVM beobachtet, welche Programmteile häufig laufen.



Warum lohnt sich das?

Das Übersetzen kostet zunächst Zeit.
Bei vielen Wiederholungen kann sich diese Arbeit auszahlen.

Danach Maschinencode nutzen

Bei weiteren Ausführungen dieser Teile nutzt die JVM den erzeugten Maschinencode.
Die CPU führt ihn direkt aus.

Die JVM kann im selben Programmlauf interpretierte und kompilierte Teile ausführen.

Erneut starten oder Quelltext ändern

Unverändert erneut starten

```
java Hallo
```

Die JVM verwendet die vorhandene Datei `Hallo.class`.
Ein erneuter Aufruf von `javac` entfällt.

Quelltext geändert

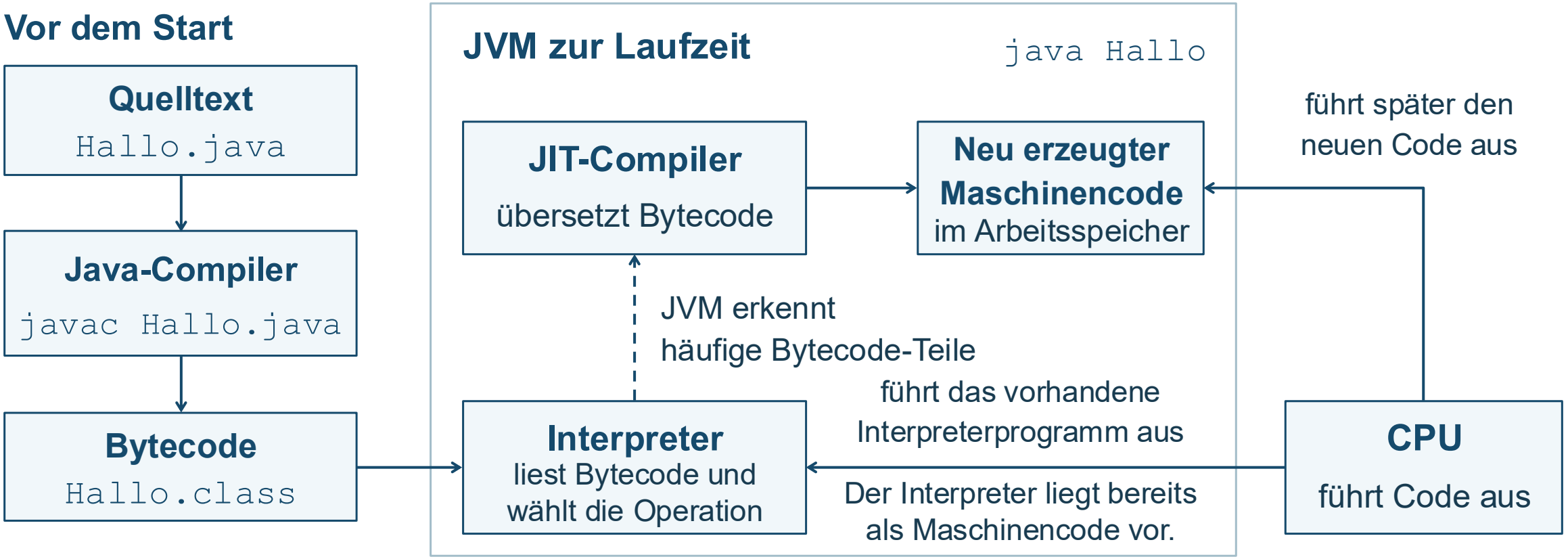
```
javac Hallo.java
java Hallo
```

Zuerst neuen Bytecode erzeugen.
Dann das Programm erneut starten.
Sonst läuft die alte `.class`-Datei.

Was passiert mit dem JIT-Maschinencode?

Er liegt normalerweise im Arbeitsspeicher der laufenden JVM.
Die gespeicherte `.class`-Datei bleibt dabei unverändert.

Java-Ausführung im Überblick



Das Programm läuft weiter, während der JIT-Compiler häufige Teile übersetzt.

Kompiliert oder interpretiert?

Sprache / übliche Umsetzung	Typischer Weg zur Ausführung
C, C++, Rust, Go	Vorab in nativen Maschinencode kompilieren
Java mit JVM	Vorab: Quelltext in Bytecode kompilieren JVM: Bytecode interpretieren und per JIT kompilieren
C# mit .NET	Vorab: Quelltext in Zwischencode (CIL) kompilieren .NET: CIL meist per JIT in Maschinencode kompilieren
Python mit CPython	CPython erzeugt intern Bytecode aus dem Quelltext und interpretiert ihn überwiegend.
JavaScript mit V8	V8 erzeugt intern Bytecode aus dem Quelltext und nutzt Interpreter und JIT-Compiler.

Python und JavaScript: Die Laufzeit erzeugt den Bytecode intern.

Oracle JDK 25 LTS einrichten

Oracle JDK 25 LTS installieren

Oracle-Downloadseite: JDK 25 LTS auswählen.
Passend zu Betriebssystem und Prozessor auswählen.

JDK = Java Development Kit

Enthält den Compiler `javac` und die JVM zum Ausführen.

```
java --version
javac --version
```

Beide Befehle: Version 25 oder höher.

Terminal nach der Installation neu öffnen.
Bei fehlendem Befehl oder älterer Version:
JDK-bin-Ordner in `PATH` aufnehmen.

Windows: `PATH` einrichten (Oracle-Anleitung)

Syntaxdiagramme für unseren Java-Einstieg

<Beispiel>



Abgerundete Kästen (Terminale)

Diese Zeichen oder Wörter schreiben wir genau so in den Quelltext.

Rechteckige Kästen (Nichtterminale)

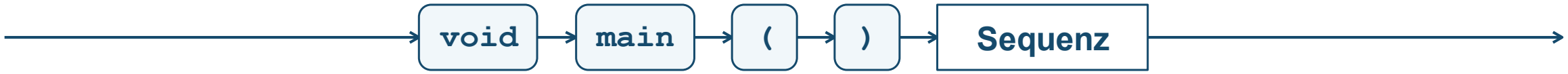
Hier setzen wir passenden Java-Code ein, z. B. einen Ausdruck wie `"Hallo"`.

Teil der Java-Grammatik

Die Diagramme beschreiben die Java-Syntax, die wir in diesem Kapitel verwenden.
Für Leerzeichen, Tabs und Zeilenumbrüche zeichnen wir keine eigenen Kästen ein.

Der Programmeinstieg: `main`

<Programmeinstieg>




```
void main() {
    IO.println("Hallo, Java!");
}
```

`main`

Hier beginnt unser Programm.

`void`

Kein Rückgabewert.

`()`

Unser Einstieg hat keine Parameter.

Eine kompakte Quelldatei genügt. Klassen und Objekte behandeln wir später.

Das erste vollständige Java-Programm

```
void main() {
    IO.println("Hallo, Java!");
}
```

Dateiname

`Hallo.java`

Ausgabe

```
Hallo, Java!
```

`{` und `}`

Begrenzen den Block mit den auszuführenden Anweisungen.

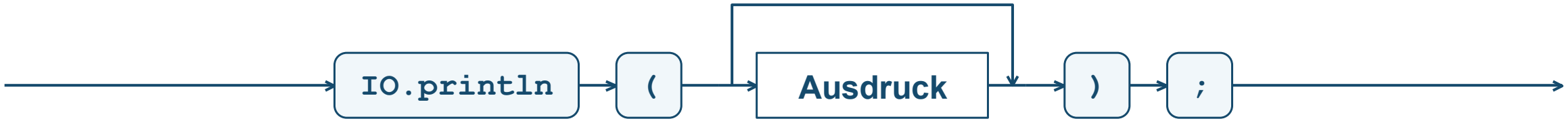
`;`

Beendet hier die Einzelanweisung der Bildschirmausgabe.

Groß- und Kleinschreibung zählt: `main`, `IO` und `println` genau übernehmen.

Bildschirmausgabe mit `IO.println()`

<Ausgabe>




```
IO.println("Hallo!");
IO.println(18);
IO.println();
```

Ein Wert erscheint als Text.
Danach beginnt eine neue Zeile.

Ohne Argument entsteht
eine leere Zeile.

Die Anführungszeichen begrenzen den Text im Code. Sie erscheinen nicht mit.

Eine Java-Datei im Terminal anlegen

```
mkdir java-start
cd java-start
```

macOS / Linux	Windows (PowerShell)
`nano Hallo.java`	`notepad Hallo.java`
Speichern: Strg+O, Enter. Schließen: Strg+X.	Code eingeben und als `Hallo.java` speichern.

Den vollständigen Code mit `void main()` eintragen.
Dateiendung `.java` verwenden, nicht `.java.txt`. Als UTF-8 speichern.

Übersetzen und starten

1 Übersetzen

```
javac Hallo.java
```

Erzeugt `Hallo.class` im selben Ordner. Bei Erfolg meist keine Meldung.

2 Ausführen

```
java Hallo
Hallo, Java!
```

Die zweite Zeile ist die Ausgabe.

Beim Start geben wir den Namen ohne `.class` an.

Ändern, erneut übersetzen, erneut starten

1 Quelltext ändern und speichern

Zum Beispiel den Text der Ausgabe ändern.

2 `javac Hallo.java` erneut ausführen

Erst dadurch entsteht der neue Bytecode.

3 `java Hallo` erneut ausführen

Die JVM startet die zuletzt erfolgreich übersetzte Version.

```
java Hallo.java
```

Alternative abgekürzte Ausführung:
übersetzt intern und startet direkt.

Nach einem Compilerfehler zuerst den Fehler beheben und erneut kompilieren.

Das erste Projekt in IntelliJ IDEA

Schritt	Auswahl
1 New Project	Java
2 Projektname	`Prog1`
3 Build system	IntelliJ
4 JDK	Version 25 (oder höher) auswählen oder über Download JDK laden
5 Create	Für den Einstieg ohne zusätzlichen Beispielcode

Ein Projekt für die gesamte Lehrveranstaltung: pro Aufgabe ein Java Compact File unter `src`. Ab Klassen und Objekten nutzen wir Packages im selben Projekt.

Java-Datei in IntelliJ schreiben und starten

1 Für jede Aufgabe eine eigene Datei

Im Ordner `src`: New → Java Compact File, Name `Hallo`.

2 Code eingeben

Den gleichen Inhalt wie in `Hallo.java` im Terminal verwenden.

3 Starten

In der gewünschten Datei das Startsymbol neben `main` anklicken und Run wählen.

4 Ausgabe ansehen

Das Fenster Run zeigt die Programmausgabe.

Run übersetzt zuerst den Quelltext und startet danach die JVM.

IntelliJ: kostenloser Einstieg und Hochschulzugang

Kostenlose Kernfunktionen

Java entwickeln, Programme ausführen und debuggen ist kostenlos möglich.

Community Edition: inzwischen gemeinsam mit Ultimate

Seit 2025.3 gibt es einen gemeinsamen IntelliJ-Installer.
Die früheren Community-Funktionen bleiben kostenlos.

Ultimate für Studierende und Lehrende

Nach Prüfung der Berechtigung ist eine kostenlose Bildungslizenz möglich.
Sie gilt für Lernen und Lehre, nicht für kommerzielle Arbeit.

Für Programmierung 1 genügen die kostenlosen Funktionen.
Antrag und Bedingungen: jetbrains.com/academy/student-pack/

Andere Editoren und Entwicklungsumgebungen

IDE = Integrated Development Environment (integrierte Entwicklungsumgebung).

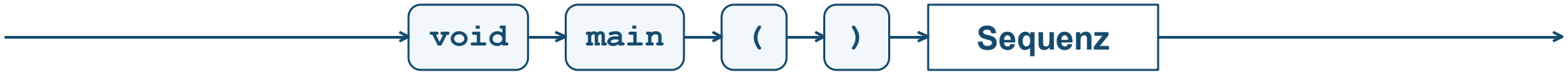
Werkzeug	Beispiele	Für Java zu beachten
Code-Editor	Visual Studio Code Sublime Text, Vim	JDK separat einrichten. Java-Unterstützung ggf. ergänzen.
IDE	IntelliJ IDEA Eclipse IDE Apache NetBeans	Editor, Übersetzen, Ausführen und Debugger sind integriert.

Sie dürfen das Werkzeug wählen. Das Java-Programm bleibt dasselbe.

Die verwendete Version muss Java 25 (oder höher) und kompakte Quelldateien unterstützen.

Der Programmeinstieg und seine Anweisungen

<Programmeinstieg>




```
void main() {
    IO.println("Hallo!");
}
```

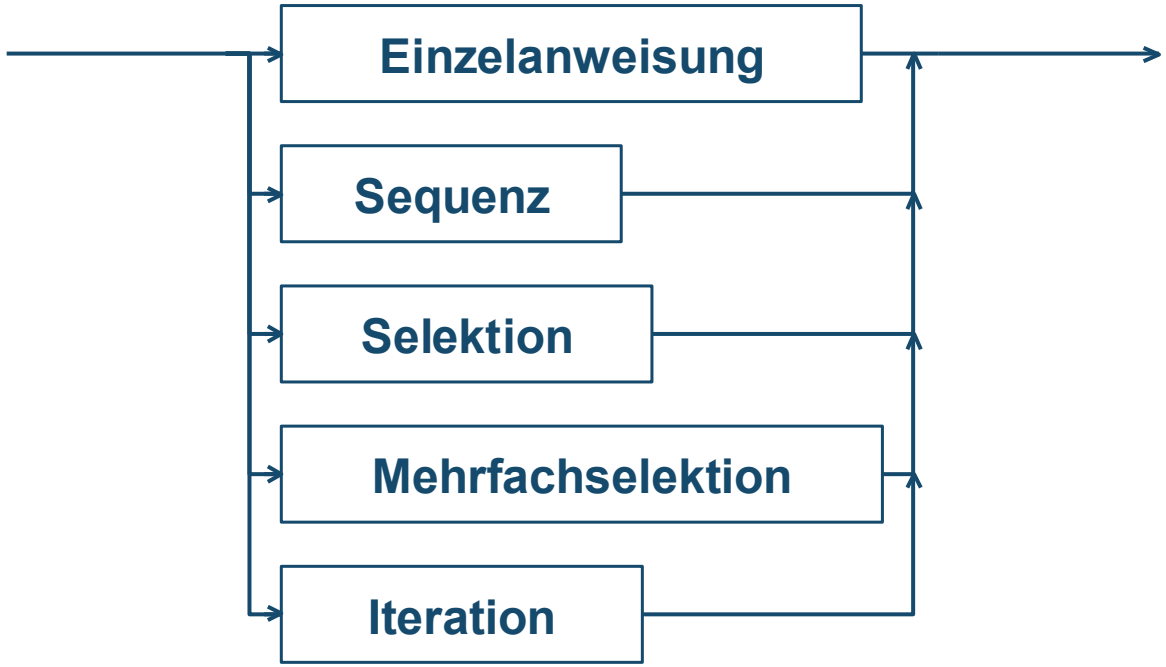
Die geschweiften Klammern begrenzen die Sequenz des Programmeinstiegs.

Für jede Anweisung darin beginnen wir beim Startsymbol <Anweisung>.

Die folgenden Diagramme beschreiben die Java-Konstrukte dieses Kapitels.

Das Startsymbol <Anweisung>

<Anweisung>



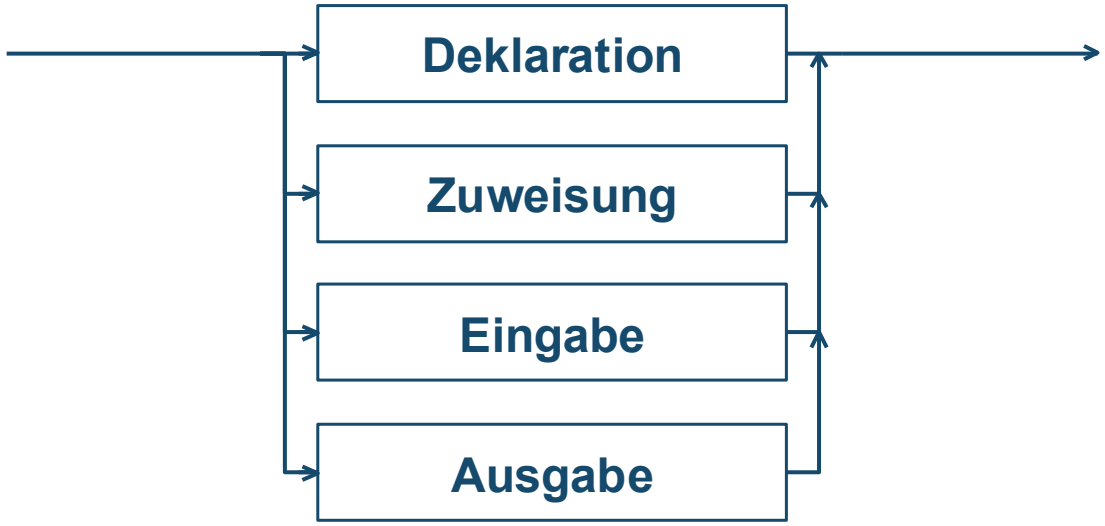
Jeder Pfad wählt einen Grundbaustein. Die Rechtecke verweisen auf die zugehörigen Syntaxdiagramme.

Eine Sequenz kann wieder Anweisungen enthalten. So lassen sich die Grundbausteine verschachteln.

<Anweisung> ist unser Startsymbol für Anweisungen innerhalb einer Sequenz.

Einzelanweisungen

<Einzelanweisung>



Variablen anlegen

```
int anzahl = 3;
```

Den Wert einer Variable ändern

```
anzahl = anzahl + 1;
```

Eine Zeile von der Tastatur lesen

```
name = IO.readln();
```

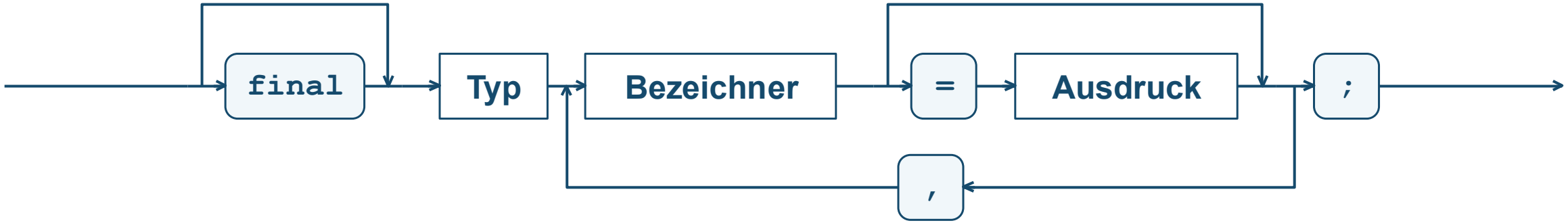
Einen Wert auf dem Bildschirm ausgeben

```
IO.println(anzahl);
```

Wir beginnen mit diesen vier bekannten Formen. Jede endet mit einem Semikolon.

Deklaration von Variablen

<Deklaration>



<Typ>: `boolean` (Wahrheitswert), `int` (ganze Zahl), `double` (Gleitkommazahl), `String` (Text)

```
int anzahl;
int a = 1, b, c = 3;
boolean bereit = true;
```

```
double preis = 2.5;
String name = "Ada";
final int MAXIMUM = 10;
```

Der Typ gilt für alle Namen der Deklaration. Jeder Anfangswert ist einzeln optional.
Vor dem Lesen muss eine Variable einen Wert haben. Mit `final` darf sie ihn nur einmal erhalten.

Zuweisung an eine Variable

<Zuweisung>



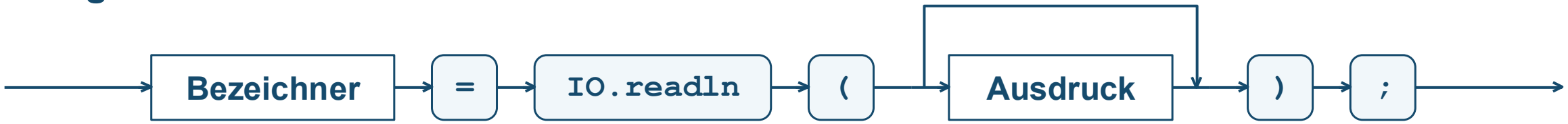
Zuerst den Ausdruck rechts auswerten, dann den Wert in der Variable links speichern.
Der Wert muss zu ihrem Typ passen. = weist zu, == vergleicht.

Schreibweise	Beispiel für `int anzahl`	Bedeutung
`=`	`anzahl = 8;`	Speichert 8 in `anzahl`.
`+=`	`anzahl += 2;`	`anzahl = anzahl + 2;`
`-=`	`anzahl -= 2;`	`anzahl = anzahl - 2;`
`*=`	`anzahl *= 2;`	`anzahl = anzahl * 2;`
`/=`	`anzahl /= 2;`	`anzahl = anzahl / 2;`
Inkrement (`++`)	`anzahl++;` oder `++anzahl;`	`anzahl += 1;` (als Anweisung)
Dekrement (`--`)	`anzahl--;` oder `--anzahl;`	`anzahl -= 1;` (als Anweisung)

Java erlaubt weitere Zuweisungsoperatoren. Vollständige Regeln: Java-Sprachspezifikation.

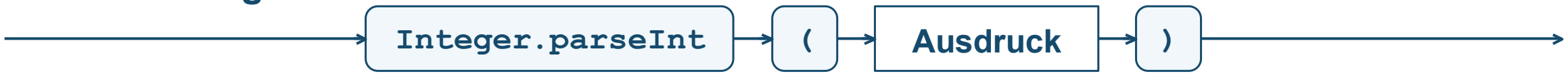
Eingabe mit `IO.readln()`

<Eingabe>



`IO.readln()` liest eine ganze Zeile und liefert einen `String`. Die Variable hat daher den Typ `String`. Ein optionaler Text in den Klammern erscheint vor der Eingabe.

<UmwandlungInInt>

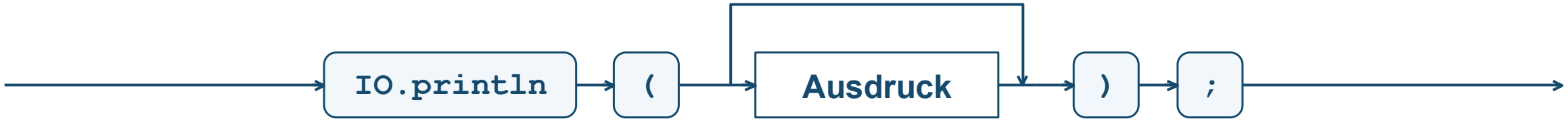



```
String name;
name = IO.readln("Name: ");
int alter = Integer.parseInt(IO.readln("Alter: "));
```

`Integer.parseInt(...)` wandelt einen Text mit einer gültigen ganzen Zahl in einen `int`-Wert um.

Ausgabe mit `IO.println()`

<Ausgabe>



`IO.println(...)` gibt den Wert des Ausdrucks aus und beginnt danach eine neue Zeile.
Der Ausdruck ist optional. Ohne Ausdruck entsteht eine leere Zeile.

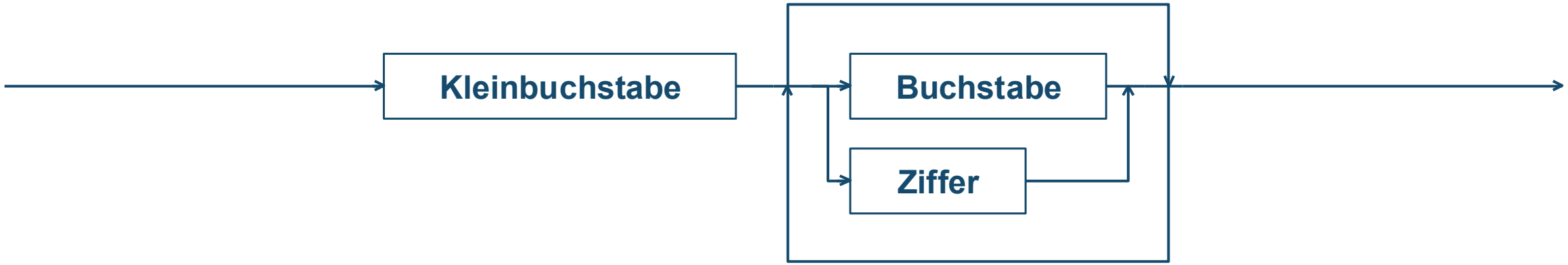
```
IO.println("Hallo!");
IO.println(2 + 3);
IO.println("Anzahl: " + anzahl);
IO.println();
```

Text, Rechenergebnis,
Text mit Variablenwert
oder eine leere Zeile.

Bezeichner: Namen für unsere Variablen

<Bezeichner>

Übliche Schreibweise für Variablennamen




```
anzahl    anzahlTeilnehmer    istBereit    wert2
```

Code Conventions: aussagekräftige Variablennamen in `lowerCamelCase`.
Konstanten: Großbuchstaben mit Unterstrichen, z. B. `MAX_WERT`.

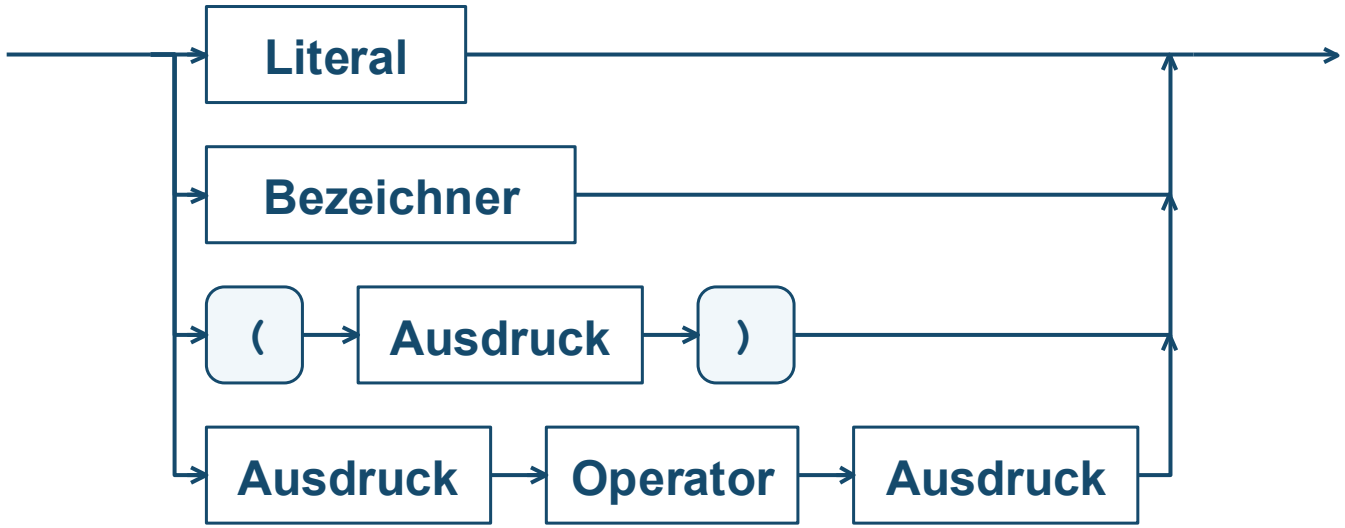
Java erlaubt darüber hinaus weitere Schreibweisen.
Die vollständigen Regeln stehen in der Java-Sprachspezifikation zu Bezeichnern.

Ausdrücke und ihre Werte

Ein Ausdruck liefert beim Auswerten einen Wert.

Dieser Wert besitzt einen Datentyp, z. B. `int`, `double`, `boolean` oder `String`.

<Ausdruck>



Für `int anzahl = 3;`:

Ausdruck	Wert
`18`	`18`
`anzahl`	`3`
`anzahl * 2`	`6`
`(anzahl + 2) * 4`	`20`

Literal: ein direkt angegebener Wert wie `true`, `18`, `2.5` oder `"Hallo"`.

Das Diagramm zeigt typische Formen. Weitere Möglichkeiten: Java-Sprachspezifikation.

Rechnen und Text verbinden

Operatoren verknüpfen passende Werte. Das Ergebnis besitzt wieder einen Datentyp.

Operator	Ausdruck	Wert (Typ)
`+` Addition	`2 + 3`	`5` (`int`)
`-` Subtraktion	`8 - 3`	`5` (`int`)
`*` Multiplikation	`4 * 3`	`12` (`int`)
`/` Division	`7 / 3` `7.0 / 3.0`	`2` (`int`) ca. `2.333` (`double`)
`%` Divisionsrest	`7 % 3`	`1` (`int`)
`+`, `-` Vorzeichen	`+3` und `-3`	`3` und `-3` (`int`)
`+` Text verbinden	`"Hallo " + "Ada"`	`"Hallo Ada"` (`String`)

Diese Ausdrücke können rechts von = oder innerhalb von `IO.println(...)` stehen.

Vergleiche und logische Verknüpfungen

Zahlen vergleichen oder `boolean`-Werte verknüpfen. Das Ergebnis ist ein `boolean`-Wert.

Operator(en)	Beispiele	Wert (`boolean`)
Gleichheit `==`, `!=`	`3 == 3` `3 != 3`	`true` `false`
Größenvergleich `<`, `<=`, `>`, `>=`	`3 < 5` `3 >= 5`	`true` `false`
Negation `!`	`!true`	`false`
UND `&&`, `&`	`true && false`	`false`
ODER `||`, `|`	`true || false`	`true`
Exklusives ODER `^`	`true ^ true`	`false`

Bei `&&` genügt links `false`, bei `||` links `true`, um die Auswertung zu beenden.
`&`, `|` und `^` werten beide Seiten aus. Vergleiche von Textinhalten folgen später.

Klammern und der Vorrang der Operatoren

Java beachtet den Vorrang der Operatoren. Klammern können ihn verändern.

Ausdruck	Wert	Typ
`2 + 3 * 4`	`14`	`int`
`(2 + 3) * 4`	`20`	`int`
`10 - 3 - 2`	`5`	`int`
`"Summe: " + 2 + 3`	`"Summe: 23"`	`String`
`"Summe: " + (2 + 3)`	`"Summe: 5"`	`String`
`true || false && false`	`true`	`boolean`
`(true || false) && false`	`false`	`boolean`

`*`, `/` und `%` binden stärker als `+` und `-`. `&&` bindet stärker als `||`.
Bei `+` und `-` auf derselben Stufe wird von links nach rechts gerechnet.

Weitere Ausdrucksformen

Auch vorgegebene Aufrufe und andere Formen können Werte liefern.

Form	Ausdruck	Wert	Typ
Einlesen	`IO.readln("Name: ")`	`"Ada"` (bei Eingabe Ada)	`String`
Text in Zahl umwandeln	`Integer.parseInt("18")`	`18`	`int`
Typumwandlung	`(int) 3.9`	`3`	`int`
Bedingter Ausdruck	`3 > 5 ? 3 : 5` `3 < 5 ? 3 : 5`	`5` `3`	`int`

`? :`: Bei `true` entsteht der Wert vor `:`, bei `false` der Wert danach.

Umwandlungen folgen genauer im nächsten Kapitel. Weitere Formen und Regeln: JLS Kapitel 15.

Ausblick: Bitoperatoren für `int`

Operator	Bedeutung	Beispiel / Ergebnis
`&`	Bitweises UND	`6 & 3` ergibt `2`
`|`	Bitweises ODER	`6 | 3` ergibt `7`
`^`	Bitweises exklusives ODER	`6 ^ 3` ergibt `5`
`~`	Alle Bits umkehren	`~6` ergibt `-7`
`<<`	Nach links schieben	`8 << 1` ergibt `16`
`>>`	Nach rechts, Vorzeichen erhalten	`-8 >> 1` ergibt `-4`
`>>>`	Nach rechts, links Nullen einfügen	`-8 >>> 1` ergibt `2147483644`

Die Operatoren verarbeiten Bits. Die Bitdarstellung folgt im nächsten Kapitel.

Vollständige Regeln: Java-Sprachspezifikation.

Einlesen, rechnen und ausgeben

```
void main() {
    String text = IO.readln("Anzahl: ");
    int anzahl = Integer.parseInt(text);
    int doppelt = anzahl * 2;

    IO.println(doppelt);
}
```

Eingabe

```
7
```

Ausgabe

```
14
```

Die Anweisungen bilden eine Sequenz: Sie laufen in dieser Reihenfolge ab.

In weiteren Beispielen zeigen wir häufig nur den Inhalt von `main`.

Kommentare im Quelltext

<Zeilenkommentar>



<Blockkommentar>




```
// Anfangswert festlegen.
int anzahl = 3;

/*
 * Dieser Kommentar geht über mehrere Zeilen.
 */
```

Kommentare erläutern den Code und erzeugen keine auszuführenden Anweisungen.

Dokumentationskommentare

<Dokumentationskommentar>




```
/**
 * Gibt eine Begrüßung aus.
 */
void main() {
    IO.println("Guten Tag!");
}
```

Steht vor einer Deklaration.
Hier: vor dem Programmeinstieg.

`javadoc` kann daraus eine Dokumentation erzeugen.

Blockkommentare lassen sich nicht ineinander verschachteln.

Sequenz: ein Block aus Anweisungen

<Sequenz>




```
{
    int a = 18;

    a = a - 12;
    IO.println(a);
}
```

Die Sequenz darf leer sein oder beliebig viele Anweisungen enthalten.

Jede Anweisung beginnt wieder bei <Anweisung>.

Die Ausführung erfolgt der Reihe nach.

Selektion mit `if` und `else`

<Selektion>




```
if (a > b) {
    a = a - b;
} else {
    b = b - a;
}
```

Bei `true` läuft der erste Zweig.
Bei `false` läuft der `else`-Zweig.

`else` ist optional. Ohne `else` geht es bei `false` hinter der Selektion weiter.

Deklarationen sind im Zweig nur innerhalb einer Sequenz mit `{` und `}` erlaubt.

Whitespace und Einrückung

Whitespace: Leerzeichen, Tabs und Zeilenumbrüche. Ohne Trennung werden zwei Wörter zu einem. `int anzahl` braucht daher Leerraum. Bei `if(` bildet die Klammer bereits eine Wortgrenze.

Variante A

```
if (a > b) {
    a = a - b;
} else {
    b = b - a;
}
```

Variante B

```
if(a>b)
{
  a=a-b;
}
else
{
  b=b-a;
}
```

Variante C

```
if (a > b) { a = a - b; } else { b = b - a; }
```

Als Entwickler bestimmen wir die Einrückung selbst. Alle drei Varianten bewirken dasselbe. Empfehlung: Java Code Conventions, damit wir fremden Code leichter lesen und verstehen.

Bedingungen in Java

<Bedingung>




```
boolean bereit = true;
int anzahl = 3;

if (bereit && anzahl > 0) {
    IO.println("Start");
}
```

Der Ausdruck muss einen `boolean`-Wert liefern.

Das kann z. B. eine Variable, ein Vergleich oder eine logische Verknüpfung sein.
Die Zahl 1 ist keine Bedingung.

Der Rumpf von Verzweigungen und Schleifen

Der Rumpf ist die Anweisung, die ein Zweig oder ein Schleifendurchlauf ausführt.
Das kann eine Einzelanweisung oder eine Sequenz mit `{` und `}` sein.

Einzelanweisung

```
if (bereit)
    IO.println("Start");
else
    IO.println("Warten");
```

Sequenz mit derselben Anweisung

```
if (bereit) {
    IO.println("Start");
} else {
    IO.println("Warten");
}
```

`;` allein ist eine leere Anweisung und bewirkt nichts.
Eine Deklaration ist als Rumpf nur innerhalb einer Sequenz mit `{` und `}` erlaubt.

Empfehlung: Den Rumpf als Sequenz mit geschweiften Klammern schreiben.

Eine Entscheidung Schritt für Schritt

```
int a = 18;
int b = 12;

if (a > b) {
    a = a - b;
} else {
    b = b - a;
}

IO.println(a);
```

Bedingung

`18 > 12` ergibt `true`.

Ausgeführter Zweig

`a` erhält den Wert 6.

Übersprungener Zweig

`b` bleibt 12.

Danach

Die Ausgabe lautet 6.

Von den beiden Zweigen läuft genau einer.

Selektionen verschachteln

Mit einem verschachtelten Block

```
if (punkte > 8) {
    IO.println("Sehr gut");
} else {
    if (punkte > 5) {
        IO.println("Bestanden");
    } else {
        IO.println("Noch einmal");
    }
}
```

Als `else if`-Kette

```
if (punkte > 8) {
    IO.println("Sehr gut");
} else if (punkte > 5) {
    IO.println("Bestanden");
} else {
    IO.println("Noch einmal");
}
```

Beide Formen verhalten sich gleich. Für 7 Punkte erscheint „Bestanden“.
Bei der Kette ist die nächste `if`-Anweisung direkt der Rumpf von `else`.

Ohne Blöcke gehört `else` zum nächsten vorherigen `if`, das noch kein `else` hat.

Wo ist eine lokale Variable sichtbar?

Sichtbarkeit: Bereich im Quelltext, in dem ein Name verwendet werden darf.

```
int ergebnis = 0;

if (true) {
    int zuschlag = 2;

    ergebnis = ergebnis + zuschlag;
}

IO.println(ergebnis);
IO.println(zuschlag); // Compilerfehler
```

`ergebnis` ist ab seiner Deklaration im umgebenden Block sichtbar.

`zuschlag` ist ab seiner Deklaration nur im inneren Block sichtbar.

Nach dessen Ende dürfen wir diesen Namen hier nicht verwenden.

Mehrfachselektion: Doppelpunktform mit `break`

```
int auswahl = 2;

switch (auswahl) {
    case 1:
    case 2:
        IO.println("A");
        break;
    default:
        IO.println("?");
}
```

`case 1:` und `case 2:` führen zu denselben Anweisungen.

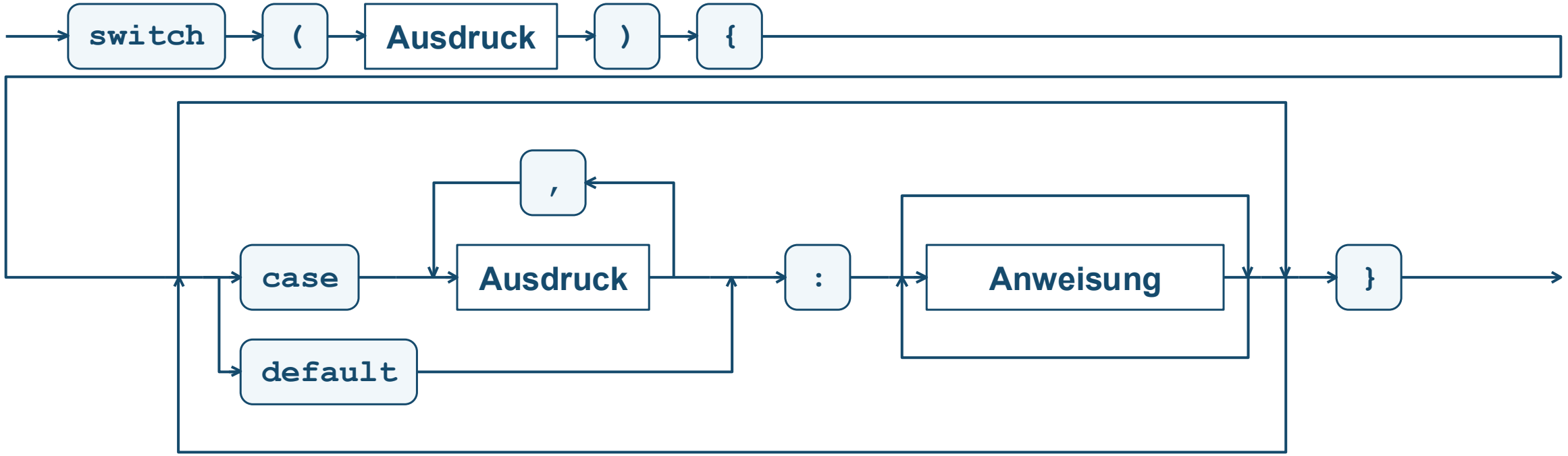
Mit `break;` endet hier `switch`.
Ohne `break;` läuft die Ausführung in die folgenden Fälle weiter, auch wenn deren Wert nicht passt.



Für `auswahl = 2`	Mit `break;`	Ohne `break;`
Ausgabe	`A`	`A`, danach `?`

Mehrfachselektion mit `switch` und `:`

<Mehrfachselektion>



Hier: Auswahlwert vom Typ `int` oder `String`. Fallwerte sind konstant, z. B. `1`, `2` oder `"Ja"`.
Fallwerte dürfen nicht doppelt vorkommen. `default` ist optional und höchstens einmal erlaubt.
Nach `:` dürfen keine, eine oder mehrere Anweisungen stehen. Leere Fälle teilen den folgenden Code.
Ein leerer `switch`-Block ist erlaubt. Weitere Formen: Java-Sprachspezifikation.

Pfeilform: Einzelanweisung oder Sequenz

Für beide Varianten gilt: `int auswahl = 2;`

Einzelanweisungen

```
switch (auswahl) {
    case 1, 2 -> IO.println("A");
    default -> IO.println("?");
}
```

Für einen Auswahlwert vom Typ `String`:

```
case "Ja" -> IO.println("A");
```

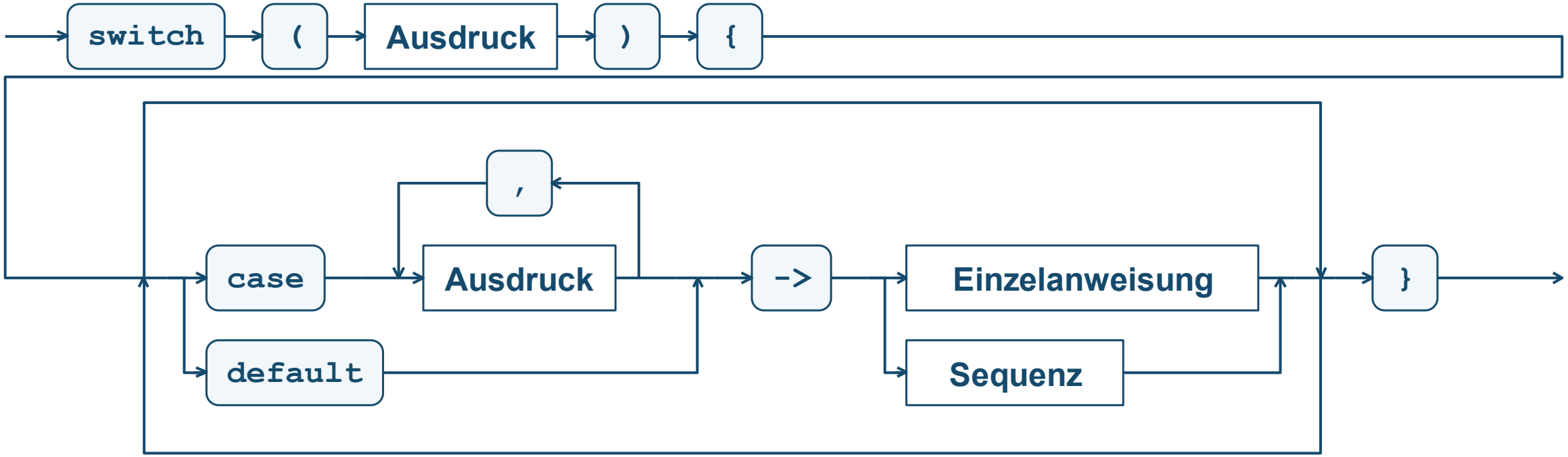
Sequenzen mit Anweisungen

```
switch (auswahl) {
    case 1, 2 -> {
        String text = "A";
        IO.println(text);
    }
    default -> {
        IO.println("?");
    }
}
```

Beide Varianten geben `A` aus. `case 1, 2` bündelt die beiden Fallwerte.
Danach geht es hinter `switch` weiter. Bei der Pfeilform ist dafür kein `break` nötig.

Mehrfachselektion mit `switch` und `->`

<Mehrfachselektion>



Auswahl und Fallwerte folgen denselben Regeln wie bei der Doppelpunktform.
Nach `->` stehen Einzelanweisungen oder Sequenzen. Deklarationen benötigen eine Sequenz.
`->` und `:` dürfen innerhalb derselben Mehrfachselektion nicht gemischt werden.
Das Diagramm zeigt die Formen für unseren Einstieg. Weitere Formen: Java-Sprachspezifikation.

Mehrfachselektion: welche Form verwenden?

Aspekt	Doppelpunktform :	Pfeilform ->
Vorteile	Bewusstes Weiterlaufen in folgende Fälle ist möglich. Bekannt aus älterem Code.	Nur der passende Zweig läuft. Kein `break` gegen Weiterlaufen in den nächsten Fall nötig.
Nachteile	Ein vergessenes `break` kann unbeabsichtigt weitere Anweisungen ausführen.	Weiterlaufen in den nächsten Fall ist nicht möglich. Gemeinsamen Code bündeln.
Typischer Einsatz	Vorhandenen Code bearbeiten oder bewusst mehrere aufeinanderfolgende Fälle ausführen.	Empfehlung für neuen Code: klare, voneinander getrennte Fälle. Mehrere Werte: `case 1, 2.`

Beide Formen können mehrere Fallwerte bündeln. Details: Java-Sprachspezifikation.

Die Iterationsformen

<Iteration>



`while`: zuerst prüfen.

`do ... while`: zuerst ausführen.

`for`: Initialisierung, Bedingung und Änderung im Schleifenkopf.

Alle drei Formen besitzen einen Rumpf, dessen Ausführung sie wiederholen.

Kopfgesteuerte Iteration mit `while`

<Kopfgesteuerteltteration>




```
int i = 1;

while (i < 4) {
    IO.println(i);
    i++;
}
```

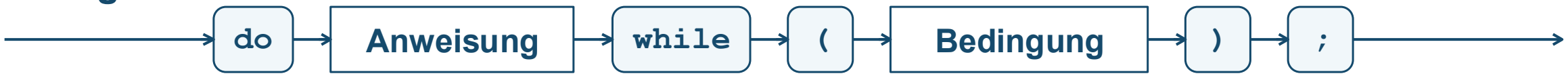
Zuerst die Bedingung prüfen.
Bei `true`: Rumpf ausführen,
dann erneut prüfen.
Bei `false`: Schleife beenden.

Ausgabe: `1`, `2`, `3`.
Drei Durchläufe, vier Prüfungen.

Ist die Bedingung schon am Anfang `false`, läuft der Rumpf kein einziges Mal.
Der Rumpf steht einmal im Quelltext. Die Schleife wiederholt seine Ausführung.

Fußgesteuerte Iteration mit `do ... while`

<FußgesteuerteIteration>




```
int zahl;

do {
    String text = IO.readln("Zahl: ");

    zahl = Integer.parseInt(text);
} while (zahl < 1);

IO.println(zahl);
```

Zuerst den Rumpf ausführen,
dann die Bedingung prüfen.
Mindestens ein Durchlauf.

Hier: Eingabe wiederholen,
solange die Zahl kleiner als `1` ist.
Das abschließende `;` gehört dazu.

Voraussetzung im Beispiel: Eingaben sind gültige ganze Zahlen im `int`-Bereich.

Die klassische `for`-Schleife

<Zählschleife>



<For-Kopf>



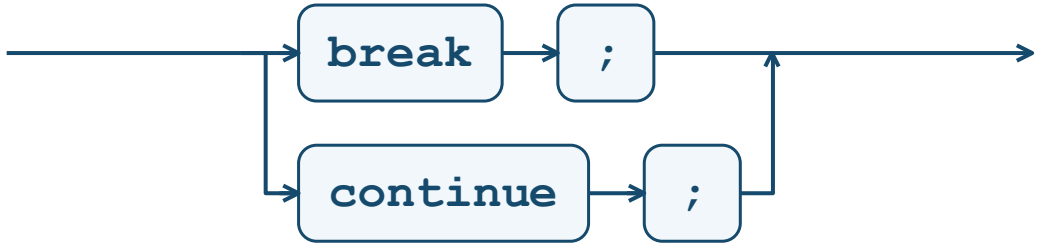

```
for (int i = 1; i < 4; i++) {
    IO.println(i);
}
```

Initialisierung genau einmal.
Dann: Bedingung, Rumpf, Änderung.
Danach wieder die Bedingung.
Ausgabe: `1`, `2`, `3`.

Alle drei Teile dürfen fehlen. Die beiden Semikolons bleiben stehen.
Ohne Bedingung gilt `true`. Ohne Initialisierung oder Änderung entfällt die jeweilige Aktion.
Das hier deklarierte `i` ist nur innerhalb dieser `for`-Schleife sichtbar.

Schleifen mit `break` und `continue` steuern

<Schleifensteuerung>



`break;` beendet die innerste umgebende Schleife oder `switch`-Anweisung.
`continue;` überspringt den restlichen Rumpf.
Bei `while` und `do ... while`: Bedingung prüfen.
Bei `for`: zuerst Änderung, dann Bedingung.

```
for (int i = 1; i < 5; i++) {
    if (i == 2) {
        continue;
    }
    if (i == 4) {
        break;
    }
    IO.println(i);
}
```

Ausgabe: `1`, dann `3`.
Bei `2` überspringen, bei `4` beenden.

Beim Umformen von `for` zu `while` darf ein `continue` die Änderung nicht überspringen.

Regeln für beide Anweisungen: Java-Sprachspezifikation.

Initialisierung und Änderung im `for`-Kopf

```
for (int i = 0, j = 4; i < j; i++, j--) {
    IO.println(i);
}
```

<Initialisierung>: Deklaration ohne abschließendes `;` oder eine Liste von Aktionen.
Eine Deklaration kann mehrere Variablen desselben Typs enthalten, hier `int i = 0, j = 4`.
Sind die Variablen schon deklariert, genügt eine Liste von Zuweisungen wie `i = 0, j = 4`.
<Änderung>: Liste von Aktionen, hier `i++, j--`. Ausführung jeweils von links nach rechts.
Eine Aktion ist hier eine Zuweisung, `++`/`--` oder ein vorgegebener Aufruf ohne `;`.

Optionale Teile	Bedeutung
`for (; i < 4; i++)`	Initialisierung erfolgt vorher.
`for (int i = 1; i < 4;)`	Der Rumpf verändert `i` oder beendet die Schleife.
`for (;;)`	Ohne Bedingung: wiederholen, bis z. B. `break;` abbricht.

Die drei Schleifenformen im Vergleich

Form	Prüfung	Rumpf läuft …	Typischer Einsatz
`while`	vor dem Rumpf	nullmal oder öfter	Wiederholen, solange eine Bedingung gilt
`do … while`	nach dem Rumpf	mindestens einmal	Eingabe mindestens einmal anfordern
`for`	vor dem Rumpf	nullmal oder öfter	Zähler oder regelmäßige Änderung im Kopf

Die Form soll den Ablauf klar zeigen. `for` braucht keine vorher bekannte Anzahl.

`for` und `while` ineinander umformen

`for`

```
for (int i = 1; i < 4; i++) {
    IO.println(i);
}
```

`while`

```
int i = 1;

while (i < 4) {
    IO.println(i);
    i++;
}
```

`for` zu `while`: Die Initialisierung kommt vor die Schleife, die Änderung ans Ende des Rumpfs.
`while` zu `for`: Die Initialisierung und die Änderung wandern in den Schleifenkopf.
Beide Programme geben `1`, `2`, `3` aus. Mit Startwert `4` führen beide den Rumpf nicht aus.

Bei `continue` muss die Änderung weiterhin stattfinden. Auch die Sichtbarkeit der Variablen beachten.

`for` oder `while` in `do … while` umformen

Ausgangsform: `for`

```
for (int i = 1;
        i < 4; i++) {
    IO.println(i);
}
```

Ausgangsform: `while`

```
int i = 1;

while (i < 4) {
    IO.println(i);
    i++;
}
```

Ziel: `do … while`

```
int i = 1;

if (i < 4) {
    do {
        IO.println(i);
        i++;
    } while (i < 4);
}
```

Beide Ausgangsformen lassen sich in das rechte Programm umwandeln. Ausgabe jeweils: `1`, `2`, `3`.
Das zusätzliche `if` prüft die Bedingung vor dem ersten Durchlauf.
Warum? Mit Startwert `4` darf nichts ausgegeben werden. Ohne `if` würde `do … while` einmal laufen.

`do … while` in `while` oder `for` umformen

Ausgangsform: `do … while`

```
int i = 1;

do {
    IO.println(i);
    i++;
} while (i < 4);
```

Ziel: `while`

```
int i = 1;
boolean erster = true;

while (erster
        || i < 4) {
    erster = false;
    IO.println(i);
    i++;
}
```

Ziel: `for`

```
int i = 1;
boolean erster = true;

for (; erster || i < 4;
        i++) {
    erster = false;
    IO.println(i);
}
```

`erster` ist der boolesche Merker: Am Anfang steht er auf `true`.
Darum ist `erster || i < 4` beim ersten Test wahr, ohne dass Java `i < 4` prüfen muss.
Im Rumpf wird `erster` auf `false` gesetzt. Ab dem nächsten Test entscheidet nur noch `i < 4`.
Alle drei geben `1`, `2`, `3` aus. Mit Startwert `4` geben alle genau einmal `4` aus.
Mit solchen Hilfsanweisungen sind die drei Schleifenformen gleich ausdrucksstark.

Schleifen im Debugger von IntelliJ IDEA

Ziel: `1, 2, 3`. Tatsächlich: `1, 2`.

```
int i = 1;

while (i < 3) {
    IO.println(i);
    i++;
}
```

Durchlauf	`i` vorher	`i` danach
1	1	2
2	2	3

1 Haltepunkt und Debug

Im Editor am linken Rand bei der Zeile `IO.println(i);` einen Haltepunkt setzen.
Neben `main` mit Debug starten.

2 Werte und Schritte

Variablen im Debug-Fenster ansehen.

Step Over: nächste Anweisung ausführen.

Resume: bis zum nächsten Haltepunkt.

3 Fehler erklären und korrigieren

Nach Durchlauf 2: `i` ist `3`.
`3 < 3` ist `false`. Für das Ziel: `i < 4`.

Die markierte Zeile steht als Nächstes an. Ihre Wirkung ist noch nicht eingetreten.

Geschachtelte Schleifen

```
for (int y = 0; y < 2; y++) {
    for (int x = 0; x < 3; x++) {
        IO.println("(" + x + "," + y + ")");
    }
}
```

Äußerer Durchlauf	Innere Schleife	Ausgaben
`y = 0`	`x = 0, 1, 2`	`(0,0), (1,0), (2,0)`
`y = 1`	`x = 0, 1, 2`	`(0,1), (1,1), (2,1)`

Zeilenweise Ausgabe: `y` legt die Zeile fest. Innen durchläuft `x` alle Punkte dieser Zeile.
Bei jedem äußeren Durchlauf startet die innere Schleife neu. Insgesamt: `2 · 3 = 6` Punkte.

Sprungmarken: `break` und `continue`

Die Sprungmarke `aussen:` benennt hier die äußere Schleife.

Mit `break aussen;`

```
aussen:
for (int i = 1; i < 3; i++) {
    for (int j = 1; j < 4; j++) {
        if (j == 2) {
            break aussen;
        }
        IO.println(i + ":" + j);
    }
}
IO.println("Ende");
```

Ausgabe: `1:1`, dann `Ende`.
`break aussen;` beendet beide Schleifen.

Mit `continue aussen;`

```
aussen:
for (int i = 1; i < 3; i++) {
    for (int j = 1; j < 4; j++) {
        if (j == 2) {
            continue aussen;
        }
        IO.println(i + ":" + j);
    }
}
IO.println("Ende");
```

Ausgabe: `1:1`, `2:1`, dann `Ende`.
`continue aussen;` setzt außen mit `i++` und dem nächsten Bedingungstest fort.

Ein Label muss eine umgebende Anweisung benennen. Bei `continue` muss das eine Schleife sein.

Der bekannte ggT-Algorithmus in Java

Algo

```
SOLANGE a ≠ b:
  WENN a > b:
    SETZE a AUF a - b
  SONST:
    SETZE b AUF b - a
  ENDE WENN
ENDE SOLANGE
GIB a AUS
```

Entsprechender Java-Ausschnitt

```
while (a != b) {
    if (a > b) {
        a = a - b;
    } else {
        b = b - a;
    }
}

IO.println(a);
```

Der Algorithmus bleibt gleich. Java verwendet andere Schreibregeln.
Voraussetzung: `a` und `b` sind positive ganze Zahlen.

Ein vollständiges Programm: `Ggt.java`

Voraussetzung: zwei positive ganze Zahlen im `int`-Bereich eingeben.

```
void main() {
    int a = Integer.parseInt(IO.readln("a: "));
    int b = Integer.parseInt(IO.readln("b: "));

    while (a != b) {
        if (a > b) {
            a = a - b;
        } else {
            b = b - a;
        }
    }

    IO.println(a);
}
```

ggT: Ablauf mit `a = 18` und `b = 12`

Zeitpunkt	`a`	`b`	Entscheidung
Start	18	12	`a != b` ist `true`, `a > b` ist `true`
Nach Durchlauf 1	6	12	`a != b` ist `true`, `a > b` ist `false`
Nach Durchlauf 2	6	6	`a != b` ist `false`, Schleife endet
Ausgabe	6	6	Der ggT ist 6

Jeder Durchlauf verringert die größere der beiden positiven Zahlen.
Dabei bleibt ihr ggT erhalten. Sobald beide gleich sind, ist das Ergebnis erreicht.

Den ggT mit mehreren Fällen prüfen

Eingabe `a`	Eingabe `b`	Erwartete Ausgabe	Was wird geprüft?
18	12	6	Erster Wert größer
12	18	6	Zweiter Wert größer
7	7	7	Kein Schleifendurchlauf
1	9	1	Ein Wert ist 1
21	14	7	Ein weiteres Zahlenpaar

0 erfüllt die Voraussetzung nicht: Bei `a = 0` und `b = 12` würde nichts kleiner.
Solche Eingaben müssten wir vor der Berechnung zurückweisen.

Fehler unterscheiden und gezielt suchen

Fehlerart	Beispiel	Wann fällt es auf?
Syntaxfehler	Semikolon oder Klammer fehlt	Beim Übersetzen
Verstoß gegen Typregeln	`int anzahl = "drei";`	Beim Übersetzen
Laufzeitfehler	`Integer.parseInt("drei")`	Bei der Ausführung
Falsche Programmlogik	Schleife läuft einmal zu wenig	Ergebnis / Ablauf passt nicht

Semantik bezeichnet die Bedeutung. Ein gültiges Programm kann das Falsche tun.

Die Grundbausteine in Java

Grundbaustein aus Kapitel 1	Java in diesem Kapitel
Einzelanweisung	Deklaration, Zuweisung, Eingabe oder Ausgabe
Sequenz	Mehrere Anweisungen im Block `{ ... }`
Selektion	`if (...) { ... } else { ... }`
Mehrfachselektion	`switch (...) { case ... -> { ... } ... }`
Kopfgesteuerte Iteration	`while (...) { ... }`
Fußgesteuerte Iteration	`do { ... } while (...);`
Zählschleife	`for (...; ...; ...) { ... }`

Was Sie jetzt können und was noch folgt

Jetzt

Kleine Java-Programme schreiben, übersetzen und ausführen.
Grundbausteine und Operatoren mit `boolean`, `int`, `double` und `String` verwenden.

Beim Prüfen

Eingaben und erwartete Ausgaben festlegen.
Abläufe im Debugger und anhand von Variablenwerten nachvollziehen.

Im nächsten Kapitel: Primitive Datentypen

Alle primitiven Datentypen im Detail: Wertebereiche und Zahlendarstellungen.
Rechnen mit verschiedenen Typen, Umwandlungen, Überlauf und Kodierungen im Detail.

Dokumentation zum Nachschlagen

Java 25 und der kompakte Programmeinstieg

openjdk.org/jeps/512

Java-Sprachregeln: Anweisungen und Ausdrücke

JLS Kapitel 14: Anweisungen / JLS Kapitel 15: Ausdrücke

Java-API: `IO` und `Integer.parseInt`

docs.oracle.com/en/java/javase/25/docs/api/

IntelliJ IDEA und Bildungslizenzen

jetbrains.com/help/idea/ / jetbrains.com/academy/student-pack/

Die Einträge sind anklickbar und führen zur jeweiligen Dokumentation.

Lernziele: die ersten eigenen Java-Programme

Das trainieren Sie in der anschließenden Übung:

Umgebung einrichten

JDK 25 und einen passenden Editor oder eine IDE installieren.

Kompilierung selbst ausführen

Ein erstes Programm im Terminal mit `javac` übersetzen und mit `java` starten.

Grundbausteine anwenden

Kleine Programme mit Variablen, Ein- und Ausgabe, Selektion und Iteration.

Programme prüfen

Danach auch in einer IDE arbeiten, Fälle testen und den Debugger einsetzen.

Ziel: einen Algorithmus selbstständig als lauffähiges Java-Programm umsetzen.